

Locality Driven State Management for In-Memory Stream Processing

Ben Trovato*
G.K.M. Tobin*
trovato@corporation.com
webmaster@marysville-ohio.com
Institute for Clarity in
Documentation
Dublin, Ohio, USA

Aparna Patel
Rajiv Gandhi University
Doimukh, Arunachal Pradesh
India

John Smith
The Thørväld Group
Hekla, Iceland
jsmith@affiliation.org

Lars Thørväld
The Thørväld Group
Hekla, Iceland
larst@affiliation.org

Huifen Chan
Tsinghua University
Haidian Qu, Beijing Shi, China

Valerie Béranger
Inria Paris-Rocquencourt
Rocquencourt, France

Charles Palmer
Palmer Research Laboratories
San Antonio, Texas, USA
cpalmer@prl.com

Julius P. Kumquat
The Kumquat Consortium
New York, USA
jpkumquat@consortium.net

Abstract

State access is increasingly the dominant cost in large-state stream processing, where per-record updates and timer-driven workloads turn random memory accesses into a memory-bound bottleneck. Existing backends often suffer from fragmented object layouts and long indirection chains, causing frequent cache misses that inflate tail latency and cap throughput. This paper presents a cache-friendly state storage design that shortens the access path and improves locality without changing stream-processing semantics or snapshot/restore compatibility. The design combines compact hash indexing with control-byte based parallel matching, an inlined key/value layout that aligns probing and key validation within the same cache line, and a lightweight storage hierarchy that organizes state by key-group and namespace to concentrate working sets. We evaluate the approach on Linux using (i) flink-state-benchmark microbenchmarks for isolated backend behavior and (ii) the benchset end-to-end suite for application-level impact. Across workloads with high update rates and window/timer pressure, results show

consistently higher throughput, lower latency, reduced cache-miss intensity, and a clearer shift away from memory-bound execution.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; *Generate the Correct Terms for Your Paper*; *Generate the Correct Terms for Your Paper*.

Keywords

stream processing, state management, cache locality, hash table

ACM Reference Format:

Ben Trovato, G.K.M. Tobin, Lars Thørväld, Valerie Béranger, Aparna Patel, Huifen Chan, Charles Palmer, John Smith, and Julius P. Kumquat. 2018. Locality Driven State Management for In-Memory Stream Processing. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Stateful stream processing systems have become a critical substrate for real-time analytics, monitoring, and decision-making. In these systems, correctness and expressiveness rely on maintaining large volumes of operator state—for example, keyed windows, aggregations, and timers—that must be accessed and updated for every incoming record. As state sizes grow and workloads become increasingly update-heavy, the execution profile shifts: compute becomes secondary, while state access dominates end-to-end throughput and tail latency.

Despite decades of optimization in stream processing engines, state access in common in-memory backends often remains memory-bound. The dominant cost is not hashing or comparison, but cache misses caused by random access patterns, fragmented object layouts, and deep chains of indirections. Each record update may trigger multiple pointer dereferences across scattered heap objects, turning high-level state operations into repeated stalls on last-level cache and DRAM. This effect is amplified in distributed deployments where large keyed state, frequent timer firing, and window maintenance concentrate pressure on the memory subsystem and increase sensitivity to locality.

This paper argues that state management should be redesigned with locality as a first-class goal, while preserving the semantics and operational guarantees that practitioners rely on. Our approach replaces pointer-heavy organization with a compact indexing structure that enables parallel matching over control metadata and keeps probing and key validation close in the memory hierarchy. We further adopt an inlined key/value layout to reduce fragmentation and improve cache-line utilization, and we streamline the access path so that routing to the right state fragment requires fewer layers of indirection.

Locality improvements in streaming are not solely a matter of making a single hash table faster. Streaming state is naturally structured by keys, key-groups, and namespaces (e.g., per-window or per-operator sub-state), and exploiting this structure is essential for concentrating working sets. Accordingly, we organize state hierarchically so that related state for a given key-group and namespace maps to small, independent sub-tables, reducing cache thrashing and improving predictability under mixed workloads. Importantly, these changes are designed to be compatible with existing state semantics and snapshot/restore workflows, enabling adoption without altering application logic.

We evaluate the design on Linux using both isolated microbenchmarks (via `flink-state-benchmark`) and an end-to-end benchmark suite (`benchset`) that exercises realistic application pipelines. Across a range of state sizes and key/namespace distributions, the results show consistent trends: higher throughput, lower tail latency, fewer cache misses, and a clearer shift away from memory-bound execution. These gains are most pronounced under high state pressure—windowed workloads with frequent updates and timer activity—highlighting the importance of cache-aware state storage for scalable, parallel stream processing.

Our contributions are:

- A locality-driven design for in-memory stream state that reduces cache-miss intensity via compact indexing and parallel matching.
- A lightweight state storage organization that shortens the access path and mitigates object fragmentation through inlined layouts.
- A namespace-aware, hierarchical state organization that improves working-set locality for distributed and parallel streaming workloads.

2 Background and Motivation

2.1 Problem Statement

Modern stream processing applications increasingly rely on large, mutable state for correctness and expressiveness. However, the dominant cost of stateful execution is often not arithmetic, but memory stalls triggered by state access: lookups and updates traverse scattered allocations, repeatedly missing in caches and amplifying tail latency. This behavior is especially pronounced when keys are high-cardinality and access patterns approximate random probes across large memory regions.

The problem is exacerbated by two structural properties of common in-memory state organizations. First, state is frequently stored in fragmented layouts with per-entry object overhead, which separates metadata from payload and forces extra cache lines to be touched during a single access. Second, the access path is layered: routing through multiple abstraction levels introduces additional indirections and branches, making state operations sensitive to pointer chasing rather than data movement.

2.2 Motivation

Streaming workloads intensify these issues because they couple high update rates with time- and window-driven maintenance. Windowed operators repeatedly read-modify-write state for the same logical key over time, while timers introduce bursts of accesses that compete for cache capacity. Under sustained throughput, this mix increases cache churn and makes performance hinge on how efficiently the state layout serves irregular, fine-grained accesses.

These observations motivate a design goal beyond a faster lookup primitive: state storage should be explicitly locality-aware. We seek compactness to reduce footprint and cache working-set size, short access paths to minimize indirections and branches, and layout choices that co-locate the information needed for probing and validation within a small number of cache lines. Finally, because stream processing systems depend on strict state semantics and operational tooling, these improvements must preserve existing correctness guarantees and remain compatible with snapshot and restore workflows.

3 Design Overview

Our design targets the dominant bottleneck in large-state stream processing: memory stalls during fine-grained state access. Rather than optimizing isolated instructions, we treat state management as a data-layout problem and redesign the storage path to minimize cache misses and indirections. At a high level, the system combines compact indexing, cache-aware probing, and a lightweight organization that leverages the natural structure of streaming state.

The first pillar is locality-driven layout with compact indexing. State entries are stored in a contiguous, allocation-efficient format, and the lookup path consults small control metadata to quickly identify candidate positions. To reduce branchy per-slot checks, the design uses control-byte based

parallel matching at a conceptual level, enabling a probe to filter multiple candidates using a small, cache-resident summary before touching payload. This shifts work from pointer chasing toward sequential, cache-friendly reads.

The second pillar is a lightweight access path that routes operations to the right state fragment with minimal layering. Streaming state is naturally partitioned by key-group and namespace (e.g., per-window or per-operator sub-state). We exploit this structure by organizing storage as a hierarchy from key-group to namespace to small sub-tables, which concentrates working sets and reduces cache thrashing under mixed access patterns. Crucially, sub-tables remain independent units, improving locality when workloads exhibit skew or when timers and window maintenance create bursts.

The third pillar is compatibility. The design preserves the observable semantics of state operations and supports existing operational workflows for snapshot and restore. From a system perspective, the redesign changes internal layout and routing decisions while maintaining deterministic behavior and compatibility with established checkpointing expectations.

We summarize the design principles as follows:

- **Locality first:** prioritize contiguous layouts and predictable access over pointer-rich structures.
- **Compact metadata:** keep control information small and cache-resident to guide probes.
- **Parallel candidate filtering:** use control-byte based parallel matching to reduce unnecessary payload touches.
- **Cache-line aligned validation:** co-locate probing and key/value validation within few cache lines.
- **Short access paths:** minimize indirections and branches in the routing and lookup pipeline.
- **Namespace-aware partitioning:** organize state by key-group and namespace into small sub-tables to concentrate working sets.
- **Semantics and snapshot compatibility:** preserve correctness guarantees and integrate with snapshot/restore workflows.

4 Challenges and Solutions

Random access and fragmentation. In-memory state access is dominated by irregular probes over a large key space, and traditional layouts amplify this cost by scattering per-entry metadata and payload across many allocations. The root cause is not the hash computation itself, but the number of cache lines touched during probing and key validation, which quickly becomes limited by last-level cache and DRAM latency. Our principle is to make the common-case probe both compact and cache-guided: consult a small, contiguous summary first, then touch payload only for likely candidates. We therefore use compact indexing with control-byte parallel matching at a high level to filter candidates in batches, and adopt a key inlined layout so that probing metadata and key validation are served from the same cache line whenever possible.

Deep object layers and long access chains. In many backends, a single state operation traverses multiple abstraction layers and pointer-heavy structures before reaching the actual value, turning each update into a series of dependent loads. This indirection chain increases branchiness, defeats hardware prefetching, and makes performance sensitive to object placement rather than workload semantics. Our principle is to shorten the access path by collapsing layers and reducing the number of mandatory pointer dereferences on the hot path. The solution is a lightweight storage organization that routes directly to compact state fragments and keeps the critical lookup/update steps within a small, contiguous set of memory regions, reducing stalls due to pointer chasing.

Underexploited namespace locality. Streaming state is naturally segmented (e.g., by windows and operator sub-state), yet generic storage often mixes namespaces or treats them as an afterthought, causing unrelated working sets to compete for the same cache capacity. The root cause is a mismatch between the logical structure of stateful operators and the physical layout: bursts from timers and window maintenance can thrash caches when many namespaces share large tables. Our principle is to align physical partitioning with the execution context so that related accesses reuse cache lines and interference is contained. We organize state as $\text{KeyGroup} \rightarrow \text{Namespace} \rightarrow \text{small sub-table units}$, so that each namespace has an independent, cache-friendly working set and the system avoids pulling cold state into cache during localized updates.

5 System Details

Compact indexing with control-byte parallel matching. The system represents each state sub-table using a compact index that separates a small, contiguous control region from the larger key/value payload region. Each probe first scans control bytes to identify candidate slots whose fingerprints match the queried key, using word-level parallel comparisons to filter multiple slots at once. This reduces branchy per-slot checks and keeps the first stage of probing mostly within cache-resident metadata. Only when a control match is found does the probe touch payload to validate the key and complete the operation.

Key inlined layout and cache-line behavior. To improve spatial locality, keys and values are stored in an inlined, contiguous layout that avoids per-entry object fragmentation and reduces pointer chasing. During lookup and update, the probe pattern is designed so that the information needed for candidate validation (key material and associated value) is typically accessed in the same cache line or adjacent cache lines. This alignment helps hardware prefetching and lowers the effective miss rate under random-access workloads. As a result, a single state operation touches fewer distinct cache lines, which is critical when throughput is limited by last-level cache and DRAM latency.

Lightweight access path via KeyGroup and Namespace routing. Stream processing engines naturally partition state by key-group for scalability and recovery, and further

segment it by namespace to represent windows and operator sub-state. The system routes each access by first selecting the key-group shard, then selecting the namespace-local fragment, and finally executing the compact probe within a small sub-table. This organization makes locality an explicit part of routing: operations that share execution context tend to land in the same small working set. By constraining the hot path to a small number of predictable steps and by avoiding deep indirection chains, the access pipeline becomes both faster and more stable under mixed workloads.

Snapshot/restore compatibility. The design preserves the observable semantics of state operations while changing internal layout choices, so it can integrate with existing checkpoint and recovery workflows. Logically, snapshotting serializes the contents of state (keys, namespaces, and values) in a deterministic form that is independent of in-memory probing artifacts. On restore, the system reconstructs the compact organization from the serialized state, re-establishing the same logical mapping while allowing physical layout to be rebuilt for locality. This approach maintains compatibility with established snapshot formats and operational tooling while enabling cache-friendly execution during steady-state processing.

6 Evaluation

6.1 Setup

We evaluate the proposed cache-friendly state storage design on Linux, focusing on how layout and access-path changes translate into end-to-end streaming performance. Our methodology combines two complementary workloads: (i) *flink-state-benchmark* microbenchmarks that isolate backend state operations and stress lookup/update patterns under controlled key and namespace distributions, and (ii) *benchset* end-to-end benchmarks that exercise realistic application pipelines where state access is interleaved with operator logic, windows, and timer activity. All experiments compare against two widely used in-memory baselines: `HashMapStateBackend` and `HeapStateBackend`.

We report throughput and latency to capture both steady-state capacity and tail behavior. To connect performance to memory-system effects, we also measure cache miss rate and provide a memory-bound breakdown that attributes stalls to the memory hierarchy. Across both workload families, we sweep state size and access intensity (e.g., working-set growth and update ratios), and vary key/namespace distributions (uniform, skewed, and mixed) to evaluate robustness under different locality regimes.

6.2 Results

Across the microbenchmarks, the cache-friendly design consistently improves the efficiency of core state operations as the working set grows. The largest gains appear in lookup-and update-heavy mixes with low locality, where baselines incur repeated cache misses due to fragmented layouts and repeated pointer dereferences. By filtering candidates using compact metadata before touching payloads, the redesigned

Table 1: Workloads used in evaluation and the primary behaviors they stress.

Workload	Level	What it stresses
flink-state-benchmark microbenchmarks	Micro	Isolated state lookups/updates; key and namespace distribution sweeps; controlled window/timer-like access intensity
benchset end-to-end benchmarks	E2E	Application pipelines; sustained update pressure; windowed state and timer-driven bursts; interaction with runtime scheduling

Table 2: Metrics reported and how they are collected.

Metric	Definition	Collection
Throughput	Records processed per second under steady-state load	Benchmark harness reports; validated by runtime counters
Latency	Per-record processing latency, including tail percentiles	Benchmark harness timestamps and percentile summaries
Cache miss rate	Miss intensity of relevant cache levels during execution	Hardware performance counters (e.g., Linux <code>perf</code>)
Memory-bound breakdown	Fraction of cycles attributable to memory stalls vs. retiring work	Counter-based top-down style analysis / stall attribution

access path reduces the number of cache lines accessed per operation and makes performance less sensitive to object placement.

In end-to-end pipelines, higher state pressure amplifies these trends. When workloads involve windowed state and timer-driven bursts, the benefits become more pronounced because repeated accesses concentrate within execution-local fragments. Compared with general-purpose in-memory baselines, the proposed organization sustains higher throughput under sustained update rates and reduces latency variability, particularly in the tail, without relying on workload-specific tuning.

We also observe that hardware-counter indicators align with the performance improvements. The proposed design lowers cache-miss intensity and shifts execution away from being dominated by memory stalls, which explains why improvements persist across different key and namespace distributions.

6.3 Discussion

The results suggest that state performance in stream processing is primarily limited by memory-system behavior rather than compute, and that reducing cache misses is an effective lever to improve both throughput and tail latency. The strongest benefits occur under windowed and timer-heavy workloads, where frequent updates and maintenance bursts repeatedly exercise the same key-group and namespace-local working sets. By shortening the access path and making probing more cache-guided, the design reduces stall-dominated execution and stabilizes performance under mixed access patterns; when locality is already high or the working set fits comfortably in cache, improvements remain but are naturally smaller.

7 Related Work

7.1 State management in stream processing

Prior work on stateful stream processing has developed rich semantics and execution mechanisms for large-scale, fault-tolerant state, including keyed operators, windowing, timers, and checkpoint/recovery protocols. Research in this area has explored how to minimize coordination overhead, reduce checkpoint cost, and improve recovery time, as well as how to manage state growth via incremental snapshots, compaction, and state TTL policies. Our work is complementary: rather than changing state semantics or recovery protocols, we focus on the steady-state performance bottleneck that increasingly dominates under high update rates—the memory behavior of fine-grained state access. This perspective connects system-level streaming concerns (windows, timers, and per-record updates) to low-level locality effects, and motivates a cache-aware redesign of the in-memory access path while preserving the established correctness contract.

7.2 Cache-friendly hash table and indexing techniques

There is a long line of work on cache- and locality-aware indexing, including compact open-addressing designs, metadata-guided probing, and vectorized or word-parallel matching to reduce branches and random memory reads. These techniques often separate small control information from payload, enabling probes to filter candidates using cache-resident summaries before touching keys and values. Related ideas also appear in SIMD-friendly dictionary structures and in data-layout optimizations that co-locate frequently accessed fields to reduce cache-line footprint. Our design draws on these general principles but adapts them to the streaming-state setting, where the access pattern is dominated by repeated lookups and updates across large, mutable working sets and where preserving snapshot/restore behavior is a hard requirement.

7.3 Distributed state backends and storage architectures

Distributed stream processors employ a range of state backend architectures, from purely in-memory representations to designs that integrate embedded stores or remote/disaggregated storage for capacity and durability. Prior work has studied trade-offs among throughput, latency, and operational properties such as elasticity, spill-to-disk behavior, and multi-tier caching. Other systems explore separating compute from storage or leveraging shared storage to simplify scaling and recovery. Our work targets a complementary point in this design space: improving the locality of in-memory state access within the execution process itself, so that per-record operations become less sensitive to cache misses and object indirection. This focus is particularly relevant in modern deployments where state remains logically large but is accessed at fine granularity, making cache behavior a first-order determinant of performance.

8 Conclusion

This paper argues that the limiting factor for large-state stream processing is increasingly the memory behavior of state access rather than the compute cost of operator logic. By making locality a first-class design goal, we show how compact indexing with parallel matching, inlined layouts, and a shortened access path can reduce cache-miss intensity while preserving state semantics and snapshot/restore compatibility. Organizing state by key-group and namespace further aligns physical layout with the execution context of windows and timers, helping isolate working sets and stabilize tail latency under mixed workloads. Our evaluation across isolated microbenchmarks and end-to-end pipelines demonstrates consistent improvements in throughput and latency trends, and provides evidence of a shift away from memory-bound execution. These results suggest that cache-aware state storage is a practical lever for scaling stateful operators in distributed and parallel streaming systems. Looking forward, we believe the same locality-first principles can be extended to multi-tier state hierarchies and adaptive partitioning policies that react to workload shifts while maintaining operational compatibility.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback.